Polymorphism

- Polymorphism means ability to take more than one form (One name – Many forms).
- Polymorphism is the ability of an object to behave differently at different situation.

Type of Polymorphism

1. Static Polymorphism
2. Dynamic Polymorphism

Static Polymorphism

- Static Polymorphism can be achieved using method overloading.
- It can be achieved with both instance and static method.
- Static Polymorphism is also called as compile time Polymorphism or early binding because java compiler is responsible to bind method calls with actual method at compile time.
- Java compiler will verify the method name first then method parameter to verify the matching method.
- Return type will not be considered.

Example:-

Ex1.java	**Compiler task**

`class Arithmetic{`
`void sum();`
`}`
 	
`class Ex1 {`
`public static void main(String[] args) {`
`Arithmetic ar=new Arithmetic();`
`}`
`ar.sum();`
`}`
`}`
	❖ Sum () method will be verified in the class.
	❖ Sum () method is not available, so compile time error will be given.

Ex1.java	**Compiler task**

`class Arithmetic{`
`void sum (int a , boolean b){}`
`}`
 	
`class Ex2 {`
`public static void main(String[] args){`
`Arithmetic ar=new Arithmetic();`
`ar.sum(10,20);`
`}`
`}`
	❖ Sum () method will be verified in the class.
	❖ Sum() method is available, so method parameter will be verified.
	❖ Method parameter is not matched/ not available, so compile time error will be given.

Ex3 .java	**Compiler task**

class Arithmetic {
void sum(**double** d){}
}
class Ex3 {
public static void main(String[] args) {
Arithmetic ar=**new** Arithmetic();
ar.sum(10.0);
}
}
	❖ Sum () method will be verified in the class.
	❖ Sum () method is available, so method parameter will be verified.
	❖ Method with parameter is matched.
	❖ Compiler will bind this method calls to actual method.

Dynamic Polymorphism

- ❖ Dynamic Polymorphism can be achieved using both method overriding and dynamic dispatch.
- ❖ It is also called as runtime Polymorphism or late binding because JVM is responsible to decide which method will be invoke based on actual object available in reference variable.
- ❖ It can be achieved only with instance method, can't be achieved using static method.

Program related details:-

1. When you are calling an instance method with super class reference variable which contain subclass object then method calls depends on the type of object available in reference variable.
2. When you are calling a static method with super class reference variable which contain subclass object then method calls depends on the type of reference variable.
3. When you are calling a method with super class reference variable which contain subclass object then method signature should be available in super class.
4. When you have variable in super class and sub class with the same name then...
   - ⇒ When you refer variable with super class reference which contain subclass object then variable will be referred from super class.
   - ⇒ When you refer variable with sub class reference which contain subclass object then variable will be referred from sub class only.
   - ⇒ When you want to refer variable of sub class, with super class reference which contains sub class object then you need to type cast reference into sub class.
5. When you refer variable with super class reference variable which contain subclass object then that variable should be available in super class.
6. Overriding concept available only with method not with variable. When you define same variable in sub class then it is variable hiding.

Programs:-

Person.java

```
class person{
void eating(){
System.out.println("person->eating()");
}
void walking(){
System.out.println("person->walking()");
}
static void sleeping(){
System.out.println("person-sleeping()");
   }
}
```

employee.java

```
class employee extends person{
void walking(){
System.out.println("employee- walking()");
}
static void sleeping(){
System.out.println("employee-sleeping()");
}
void working(){
System.out.println("employee-working()");
   }
}
```

student.java

```
class student extends person{
void walking(){
System.out.println("student->walking()");
}
static void sleeping(){
System.out.println("student->sleeping()");
}
void reading(){
System.out.println("student->reading()");
  }
}
```

OOPS381.java

```
class OOPS381{
public static void main(String args[]){
person pob=null;
pob=new student();
pob. eating();
pob=new employee();
pob. eating();
   }
}
//No Polymorphism
```

OOPS382.java

```
class OOPS382{
public static void main(String args[]){
person pob=null;
pob=new student();
pob. walking();
pob=new employee();
pob. walking();
  }
}
```

OOPS383.java

```
class OOPS383{
public static void main(String args[]){
person pob=null;
pob=new student();
pob. sleeping();
pob=new employee();
pob. sleeping();
 }
} //No Polymorphism
```

OOPS384.java

```
class OOPS384{
public static void main(String args[]){
person pob=null;
pob=new student();
pob. reading();
pob=new employee();
pob. working();
 }
}
```

OOPS385.java

```
class OOPS385{
public static void main(String args[]){
person pob=null;
pob=new student();
student stu=(student) pob;
stu. reading();
pob=new employee();
employee emp=(employee)pob;
emp. working();
   }
}
```

OOPS386.java

```
class OOPS386{
public static void main(String args[]){
A h=new B();
System.out.println(h. x);
  }
}
class A{
int x=10;
}
class B extends A {
String x="IND";
}
```

OOPS387.java

```
class OOPS387{
public static void main(String args[]){
A aobj=new B();
aobj. x=99;
System.out.println(aobj. x);
  }
}
class A {
int x=10;
}
class B extends A {
String x="IND";
}
```

OOPS388.java

```
class OOPS388{
public static void main(String args[]){
B bobj=new B();
bobj. x=99;
System.out.println(bobj. x);
  }
}
class A{
int x=10;
}
class B extends A{
String x="IND";
}
```

OOPS389.java

```
class OOPS389{
public static void main(String args[]){
B bobj=new B();
bobj. x="hello";
System.out.println(bobj. x);
  }
}
class A{
int x=10;
}
class B extends A{
String x="IND";
}
```

OOPS390.java

```
class OOPS390{
public static void main(String args[]){
B bobj=new B();
((A)bobj).x=99;
//A ob=(A)bobj; ob.x=99;
//Both statement are  same
System.out.println(bobj.x);
  }
}
class A{
int x=10;
}
class B extends A{
String x="IND";
}
```

OOPS391.java

```
class OOPS391{
public static void main(String args[]){
A aobj=new B();
aobj.x="Hello";
System.out.println(aobj.x);
  }
}
class A{}
class B extends A{
String x="IND";
}
```

Program output /error and details:-

Prog No.	output	error and details
OOPS381	Person->eating() Person->eating()	Here, eating () method is available in only super class so there are no overriding so that super class statement of eating () method will display.
OOPS382	student->walking() employee->walking()	Here, walking () method is available in super class and sub class with same signature so there is method overriding.
OOPS383	person->sleeping() person->sleeping()	Here, no overriding so that super class statement of sleeping () method will display.
OOPS384	Compilation error	[can't find symbol] Because, method reading () of student and method working () is not available in super class.
OOPS385	student->reading() employee-working()	Here, you want to method of sub class, with super class reference which contains sub class object then you need to type cast reference into sub class.
OOPS386	10	Here, A is super class and B is sub class then refers point 4 and 4.1.
OOPS387	99	You have variable in super class and sub class with the same name and also in main class with reference aobj and value is int type so it displays 99.
OOPS388	Compilation error	[incompatible type] You have variable in super class and sub class with the same name and you refer variable with sub class reference which contain subclass object then variable will be referred from sub class only and sub class variable is string type.
OOPS389	Hello	Details same as 388
OOPS390	IND	Refer page 137 point 4.3 for details
OOPS391	Compilation error	[can't find symbol] Because, When you refer variable with super class reference variable which contain subclass object then that variable should be available in super class.

Write your note here.....